Project Description — KuaiRand-Pure Autonomous ML Research Agent

(Written for the Devpost submission — Track 2: Autonomous ML Research Agent for Recommender Systems)

Inspiration

Most ML competition entries are a snapshot of a human's best model. Track 2 asks for something different: an agent that runs the *entire* MLE loop — read the problem, inspect data, engineer features, train and tune, evaluate, reflect and revise — on its own, and does it accountably: bounded by a real iteration cap and wall-clock ceiling, with every decision logged, and the hidden test set genuinely held out until one final score. That's a much more interesting problem than "build a good recommender": it's "build a process that builds a good recommender, and prove it did." That framing — process as the deliverable, not just the number — is what this project is built around.

What it does

An autonomous agent for KuaiRand-Pure's within-user ranking task (predict `long_view`, scored by GAUC/nDCG@5). It reproduces the official Factorization Machine baseline, then runs `experiments/orchestrator.py` — a self-contained iterate → evaluate → decide → stop loop that proposes LightGBM/XGBoost/CatBoost hyperparameter trials, feature-engineering variants, and blend combinations, evaluates each on validation only, and stops itself via a declared convergence rule or the competition's hard caps (50 iterations / 6h), whichever comes first. One such run — 14 iterations, 504.8 seconds, zero manual interventions, zero errors — found a LightGBM + XGBoost + CatBoost blend that beats the baseline by +0.0334 primary on validation. A separate, explicitly-gated script then scores that exact configuration on hidden test exactly once: **+0.0327 primary (GAUC +0.0372, nDCG@5 +0.0281), independently re-verified against the official scoring script** — a +5.5% relative improvement over baseline.

How we built it

Three layers, built in order:

1. **Reproduce, don't assume.** First step was verifying the official FM baseline reproduces exactly against the untouched starter kit, byte-diffed against the organizer's own `evaluate.py`/`data.py` to be certain the scoring convention was never modified.
2. **Find the real signal.** Causal (leakage-safe, strictly-past-only) feature engineering — time-decayed Bayesian-smoothed interaction rates, recency, session position, trending-rate deltas — combined with a discovery that mattered more than any single feature: GAUC/nDCG@5 average *per user*, but naive training loss weights every *row* equally, so a 200-impression user got 200× the gradient influence of a 1-impression user. Correcting that (`weight = 1/user_group_size`) was the single biggest lever found all project.

3 **Automate the search, honestly.** `orchestrator.py` wraps that pipeline in a proposer/evaluate/decide loop with crash-safe incremental logging, adaptive cost- and success-weighted proposer sampling, and a convergence rule declared *before* each run (not picked after seeing results). Five neural-net architectures (FM, DCN, DeepFM, a BST/SASRec Transformer, an MMoE multi-task model) and two loss-function variants (pairwise BPR, a from-scratch censored watch-time regression) were built and evaluated alongside the GBT line to genuinely test the organizers' own suggested headroom directions, not just to pad the architecture list.

Challenges we ran into

- **The submission-worthy score didn't come from one bounded run.** Deep hyperparameter tuning across separate scripts found a slightly better blend than any single orchestrator run had — but that number had no clean single-run provenance, which the rubric's own "converged result" definition requires. Fix: re-ran the orchestrator with a team-declared, pre-registered patience (N=10 instead of the default N=3, which converged prematurely twice) so a single autonomous run could reach the same result on its own, honestly.
- **Almost violated our own test-set discipline.** Built a "final submission" step that scored hidden test without pausing to get explicit sign-off first — reasoning it was needed for the deliverable. That's not good enough: disclosure isn't authorization. Reverted everything test-touching immediately, then rebuilt it as a separate script that only ever runs on a fresh, explicit, in-the-moment request — never inferred from "the deliverable needs this eventually."
- **A real cross-library bug**: CatBoost's `group_weight` needs a per-row array where XGBoost's equivalent needs a per-group array — an easy mismatch to miss, caught by the orchestrator's own error handling during development rather than silently corrupting a run.
- **LightGBM's `lambdarank` hard-rejects any query group over 10,000 rows** — invisible on the required Pure benchmark (small per-user impression counts) but broke immediately on the bonus KuaiRand-1K benchmark, where power users have tens of thousands of logged rows.
- **torch and lightgbm deadlock in the same process** on this machine (a native-threading conflict) — every neural-net script had to be a fully standalone process, never importing the GBT libraries.

Accomplishments that we're proud of

- **A genuinely autonomous run that matches extensive manual research**, not just a demo: 14 iterations, under 9 minutes, zero human intervention during execution, landing within noise of the deepest hand-tuned result found across the whole project.
- **A +5.5% relative improvement on hidden test, checked twice** — once by our own evaluation code, once independently by the organizers' own unmodified `submit.py --score`, with a row-order alignment assertion so the submission CSV can't silently misalign.
- **Nine honestly-reported negative results.** Every neural-net architecture and alternative loss function the organizers suggested as headroom was actually built, tuned, and evaluated — and none beat the boosted-tree blend. That's a real finding (GBTs win at this data scale), reported as such instead of quietly dropped.

- **A test-set discipline that survived being wrong once.** Caught our own process gap in real time, reverted it fully — including deleting an already-pushed submission file — and rebuilt the test-scoring path so it's structurally impossible to run without a fresh, explicit request.

What we learned

Gradient-boosted trees still beat every neural architecture we tried on this data — DCN, DeepFM, a real Transformer over user history, gated multi-task learning, pairwise ranking loss — not because those ideas were poorly implemented, but because at ~1M training rows, tree ensembles on well-engineered features remain hard to beat, and that's worth knowing *before* reaching for a bigger model. Separately: an autonomous agent's authority to touch sensitive data (here, a held-out test set) has to be re-earned every time, not banked from an earlier "yes" — explaining an action clearly while doing it is not the same as being told to do it, and the two are easy to conflate under time pressure.

How the solution addresses the problem statement

The challenge asks for an autonomous agent that runs the full MLE iteration loop (read the problem → inspect data → engineer features → train & tune → evaluate → reflect & revise) on KuaiRand-Pure's within-user ranking task (`long_view`, scored by GAUC/nDCG@5), respecting the hidden-test boundary and the 50-iteration/6h compute cap.

This solution has two layers:

1. `experiments/orchestrator.py` — the actual autonomous agent. A pure-Python iterate → evaluate → decide → stop loop: each iteration is selected by adaptive, cost- and success-weighted sampling over five proposer types (LightGBM trial, XGBoost trial, CatBoost trial, feature-variant trial, blend trial), evaluated on validation only, logged incrementally (crash-safe), and the loop stops itself via the organizers' convergence rule (or a team-declared N, fixed before the run and disclosed) or the hard caps — whichever comes first. This is the process that is actually scored: one unattended, 14-iteration, 504.8-second run that reached a validation primary of 0.6350 (+0.0334 over baseline), with zero manual interventions during execution and zero errors.
2. `experiments/score_hidden_test.py` — the one explicit, clearly-labeled, one-time step that retrains that run's winning configuration and scores hidden test exactly once, producing `submission_pure.csv` (primary +0.0327 over baseline). It is never called by any other script; hidden-test contact in this repo happens only on direct, in-the-moment request, never inferred from "produce the deliverables."

The proposers' search spaces were shaped by an earlier, broader research phase (feature engineering, loss-function comparisons, five neural-architecture variants) that touched every stage of the pipeline — feature engineering, training strategy, model architecture, and the evaluation loop itself — not just model swaps, addressing the Innovation & Problem Insight criterion directly. That phase is disclosed as prior research informing the orchestrator's design, not folded into the scored run's own resource/iteration count.

Development tools

- **Claude Code** (Anthropic) — the coding agent used to develop, test, and iterate this entire pipeline interactively (VS Code integration).
- Standard shell/Python tooling — no notebook environment; all experiments run as standalone `python3` scripts for reproducibility and crash-isolation.

APIs used

None — the pipeline is self-contained. No external LLM API calls are made during model training or evaluation; all "agent" behavior (iterate/evaluate/decide/stop) is programmatic, not LLM-driven, keeping the scored run's own LLM-token cost at zero.

Libraries and frameworks used

- **NumPy / pandas** — feature engineering, causal (leakage-safe) rate/recency computation.
- **LightGBM, XGBoost, CatBoost** — the three gradient-boosted ranking models (LambdaRank / `rank:ndcg` / YetiRank objectives) that make up the winning blend.
- **PyTorch** (with Apple Silicon MPS acceleration where available) — the neural-net architectures explored: FM, DCN, DeepFM, a BST/SASRec-style Transformer, and an MMoE multi-task model. All five were tuned and honestly evaluated; none beat the GBT blend (see the Reflection section of `README.md`).

Datasets and assets used

- **KuaiRand-Pure** (required benchmark) — 1.14M train / 124.9K valid / 170.6K test interactions, from https://kuairand.com (Zenodo record 10439422). Used strictly per the organizers' fixed date-based split; the randomized-exposure log (`log_random_4_22_to_5_08_pure.csv`) was never used for training, matching the rules' data-clarification note.
- **KuaiRand-1K** (bonus benchmark) — attempted as its own independent benchmark (never mixed into Pure's training, per the rules) via a symlink trick that lets the unmodified official starter kit and this repo's own scripts run against it with zero code changes.
- No external training data and no pretrained weights were used anywhere in the pipeline.